

Activation Functions

Introduction to Activation Functions

Activation functions are essential in neural networks, introducing non-linearity and enabling models to learn complex patterns. Without them, neural networks would be limited to linear transformations, incapable of solving real-world problems like image recognition, language translation, or speech synthesis.

What Happens Without Activation Functions?

- **Linear Outputs Only:** Neural networks without activation functions are purely linear models. Regardless of the number of layers, the output is a weighted sum of inputs.
 - **Limitation:** Such networks cannot model non-linear relationships (e.g., the XOR problem).

Why Activation Functions?

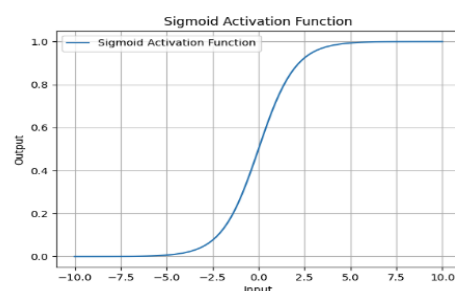
- **Non-Linearity:** They introduce non-linearity, enabling neural networks to solve complex problems like image classification or natural language processing.
- **Feature Learning:** Facilitate learning intricate patterns and relationships in data.
- **Gradient Flow:** Essential for backpropagation, allowing effective learning in deep networks.

Popular Activation Functions :

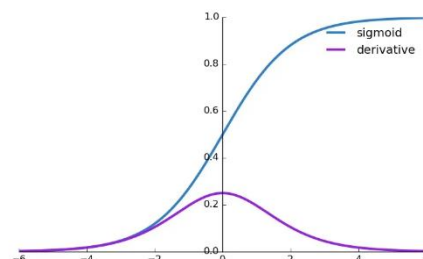
1. Sigmoid Function

- **Definition:** The sigmoid function compresses input values into a range between 0 and 1, making it ideal for binary classification tasks.
- **Equation & Graph :**

$$f(x) = \frac{1}{1+e^{-x}}$$



- **Range:** (0, 1)
- **Derivative:** $f'(x) = f(x)(1 - f(x))$
- **Key Properties:**
 - Outputs values between 0 and 1.
 - Smooth and differentiable.
 - Often used in the output layer for binary classification tasks.



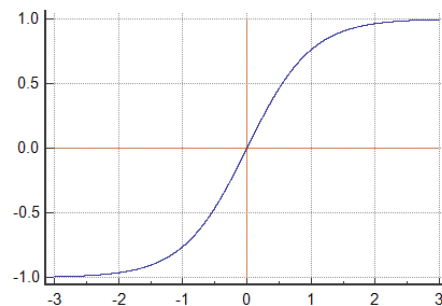
- **Advantages:**
 - Converts complex inputs into probabilities.
 - Easy to interpret results for classification tasks.
 - **Disadvantages:**
 - Suffers from the vanishing gradient problem.
 - Outputs are not zero-centered
-

2. Tanh (Hyperbolic Tangent) Function

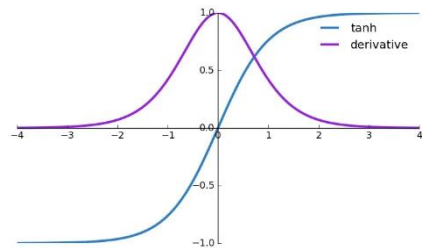
- **Definition:** The Tanh function maps input values into a range between -1 and 1. It is zero-centered, making it suitable for hidden layers in neural networks.

- **Equation & Graph :**

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



- **Range:** (-1, 1)
- **Derivative:** $f'(x) = 1 - f(x)^2$
- **Key Properties:**
 - Outputs range from -1 to 1.
 - Zero-centered output facilitates optimization.
- **Advantages:**
 - Handles both positive and negative values effectively.
 - Provides stronger gradients compared to Sigmoid.
- **Disadvantages:**
 - Suffers from the vanishing gradient problem.

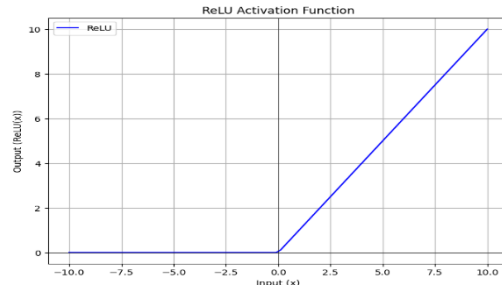


3. ReLU (Rectified Linear Unit)

- **Definition:** ReLU outputs the input directly if it is positive; otherwise, it outputs zero. It is widely used in hidden layers of deep learning models.

- **Equation & Graph :**

$$f(x) = \max(0, x)$$



- **Range:** $[0, \infty)$
- **Key Properties:**
 - Outputs 0 for negative inputs and the input itself for positive values.
 - Computationally efficient and helps deep networks converge faster.
- **Advantages:**
 - Solves the vanishing gradient problem in deep networks.
 - Easy to implement and computationally efficient.
- **Disadvantages:**
 - Can result in "dying neurons" (neurons stuck at zero).

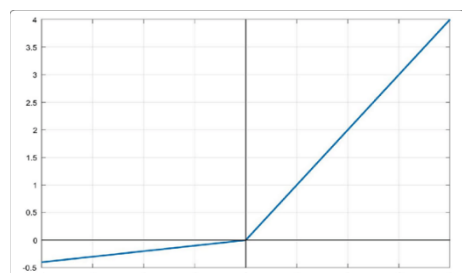
4. Leaky ReLU

- **Definition:** Leaky ReLU is a variation of ReLU that allows small negative outputs for negative inputs instead of zero, addressing the "dying ReLU" problem.

- **Equation & Graph :**

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ ax & \text{if } x \leq 0 \end{cases}$$

, where a is a small constant eg: $a=0.01$



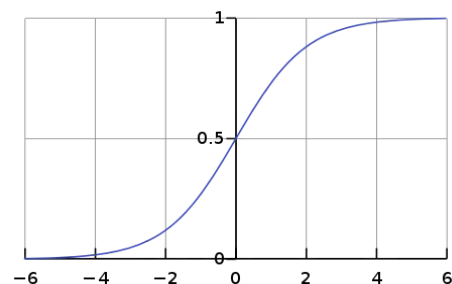
- **Range:** $(-\infty, \infty)$
- **Key Properties:**
 - Outputs positive values as-is and a small fraction of negative values.
- **Advantages:**
 - Prevents neurons from "dying" during training.
 - Allows gradient flow for negative inputs.

- **Disadvantages:**
 - Slightly more computationally expensive than ReLU.
-

5. Softmax Function

- **Definition:** The softmax function converts raw scores (logits) into probabilities that sum up to one, making it suitable for multi-class classification tasks.
- **Equation:**

$$f(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$



- **Range:** (0,1), with outputs summing to 1.
- **Key Properties:**
 - Converts raw outputs into probabilities that sum up to one.
 - Each output represents the probability of belonging to a specific class.
 - Ensures all outputs are positive and interpretable as probabilities.
- **Use Case:** Primarily used in the output layer for multi-class classification tasks where classes are mutually exclusive (e.g., digit recognition).
- **Advantages:**
 - Provides a well-defined probability distribution for each class .
 - Makes predictions interpretable by converting logits into probabilities .

Example: For raw outputs $z=[2.5,-0.3,1.8]$, softmax transforms them into probabilities like $[0.65,0.10,0.25]$.

Conclusion :

Activation functions are essential for introducing non-linearity into neural networks, enabling them to model complex patterns in data. Each activation function has its unique properties suited for specific use cases:

Activation Function	Definition	Range	Use Case	Advantages	Disadvantages
Sigmoid	Compresses input into (0,1), ideal for probabilities	(0,1)	Binary classification	Smooth gradient; outputs probabilities	Vanishing gradient; not zero-centered
Tanh	Maps input into (-1,1), zero-centered	(-1,1)	Hidden layers	Zero-centered; stronger gradients	Vanishing gradient
ReLU	Outputs input if positive; else zero	$[0, \infty)$	Deep learning hidden layers	Computational efficiency; avoids vanishing gradients	Dying neurons
Leaky ReLU	Allows small negative outputs	$(-\infty, \infty)$	Alternative to ReLU	Avoids dying neurons	Slight computational overhead
Softmax	Converts logits into probabilities	(0,1)	Multi-class classification	Probabilistic output; interpretable	Sensitive to outliers; computational cost

Choosing the right activation function depends on the task at hand and the architecture of the neural network!